

INSTITUTO FEDERAL DO ESPÍRITO SANTO

Aprimoramento da Detecção de Armas de Fogo com YOLOv8:
Uma Abordagem de Otimização de Hiperparâmetros e Arquitetura

LEANDRO BALDAN FERREIRA

VITÓRIA, ES

2024

RESUMO

A crescente violência armada no Brasil intensificou a necessidade de soluções eficazes em segurança pública, destacando a importância de sistemas de videomonitoramento inteligente capazes de detectar atividades suspeitas em tempo real. Este artigo apresenta uma pesquisa que utiliza a arquitetura YOLOv8 para desenvolver e otimizar modelos de detecção de armas de fogo em ambientes urbanos. A coleta de dados foi realizada nos campi do IFES - Guarapari e Vitória, resultando em um dataset robusto e diversificado, utilizado para treinar três variantes do modelo: uma versão tradicional (Modelo 1), uma com hiperparâmetros otimizados via Optuna (Modelo 2) e outra com modificações na *backbone* para melhorar a extração de características (Modelo 3). Os resultados demonstraram que os Modelos 2 e 3 superaram o Modelo 1 em precisão e *recall*, com o Modelo 3 alcançando alta eficiência e adaptabilidade, mesmo com tamanho reduzido. A utilização de otimização foi crucial para manter o desempenho competitivo dos modelos menores, viabilizando sua aplicação em sistemas com recursos computacionais limitados. Dessa forma, a pesquisa não apenas avança o campo da detecção de objetos, mas também estabelece uma base sólida para soluções mais eficazes e ágeis em segurança pública, permitindo implementações práticas em sistemas de videomonitoramento em tempo real.

1. INTRODUÇÃO E OBJETIVOS

A crescente demanda por soluções de segurança pública eficientes e rápidas impulsionou o desenvolvimento de tecnologias de detecção visual aplicadas à vigilância em tempo real (Miranda, 2012). A detecção de armas de fogo em imagens e vídeos, em particular, tornou-se uma área de intensa pesquisa devido ao aumento significativo de crimes envolvendo essas armas, especialmente no Brasil, onde a violência armada apresenta uma tendência crescente (Coelho, 2024). Dados mostram que armas de fogo são utilizadas em grande parte dos crimes violentos, como assaltos, furtos, invasões e homicídios (IPEA, 2024). Em 2018, o Brasil foi classificado como o 9º país mais violento no ranking global da Organização Mundial da Saúde (OMS) (Gomes, 2019), com um

número alarmante de óbitos decorrentes de armas de fogo. Esse cenário ressalta a necessidade urgente de estratégias eficazes e tecnologias de ponta para enfrentar a escalada da violência.

Sistemas de videomonitoramento inteligente emergem como uma das soluções potenciais para colaborar com o bom funcionamento da segurança pública (Nikouei et al., 2018). Esses sistemas são capazes de detectar ações suspeitas, como aglomerações, pessoas correndo ou a presença de armas de fogo, utilizando aplicações analíticas que capturam imagens e/ou sons e realizam análises em busca de situações críticas. Um Espaço Inteligente (EI) pode ser considerado um ambiente físico equipado com uma rede de sensores e atuadores, juntamente com serviços de computação que processam as informações coletadas, permitindo uma interação eficaz com os usuários e a automação de respostas no próprio espaço (do Carmo, 2021).

A aplicação do reconhecimento de objetos em sistemas de videomonitoramento é fundamental para criar ambientes inteligentes que possam responder rapidamente a situações de risco (Loureiro, 2021). Para isso, é necessário que os objetos de interesse, como armas de fogo, estejam visíveis para que as câmeras possam detectá-los de forma eficaz. Nesse contexto, este trabalho tem como objetivo a adaptação e otimização da arquitetura YOLOv8 para o reconhecimento de armas de fogo em um ambiente inteligente com videomonitoramento. A pesquisa busca aprimorar a precisão e reduzir o tamanho da rede, adequando-a para aplicações em tempo real. Utilizando o framework de otimização Optuna, realizamos ajustes na quantidade de camadas e em hiperparâmetros como número de filtros, número de vezes que a camada se repete, tamanho do kernel, tipo de módulos de rede e seus atributos.

Adicionalmente, a integração de algoritmos genéticos para encontrar a melhor configuração da arquitetura YOLO pode resultar em um modelo mais robusto e eficaz para detecção em ambientes complexos (Fan, 2024). Já a adição de outras redes convolucionais para aprimorar a extração de características foi discutida por Junos et al. (2020). O resultado é um modelo que equilibra eficiência e custo computacional, capaz de operar em sistemas de segurança que exigem respostas ágeis e precisas em ambientes complexos, contribuindo assim para a segurança pública. Assim, o trabalho tem como objetivo otimizar a arquitetura da YOLO para a aplicação em segurança pública e no reconhecimento de armas de fogo, buscando soluções que atendam à demanda por um

sistema de reconhecimento eficaz e adaptado ao contexto de videomonitoramento inteligente.

2. TRABALHOS RELACIONADOS

2.1 YOLOv8

Avanços recentes em tecnologia têm impulsionado pesquisas em sistemas automatizados para segurança pública, com um foco crescente na detecção de armas de fogo em ambientes urbanos (COELHO, 2024). As abordagens tradicionais para vigilância e identificação de ameaças, como monitoramento manual e sistemas baseados em sensores, enfrentam limitações significativas em termos de escalabilidade, precisão e capacidade de resposta a eventos em tempo real (S. AL-E'MARI, 2024).

Com a ascensão do aprendizado profundo, técnicas de visão computacional e aprendizado de máquina estão sendo cada vez mais empregadas para melhorar a eficácia da segurança pública. Modelos como as Redes Neurais Convolucionais (CNNs) e a arquitetura YOLO (You Only Look Once) têm mostrado um grande potencial na identificação e classificação de objetos, incluindo armas de fogo, diretamente de imagens capturadas por câmeras de vigilância (VIJAYAKUMAR & VAIRAVASUNDARAM, 2024). A aplicação dessa tecnologia em um Espaço Inteligente (EI) permite que uma rede de sensores e atuadores trabalhe em conjunto para monitorar ambientes, capturando e analisando dados em tempo real.

A YOLOv8 trouxe avanços significativos em termos de eficiência e precisão (Ultralytics, 2020). Ela otimiza a detecção de objetos ao combinar as melhores práticas de modelos anteriores com inovações que aumentam sua capacidade de operar em ambientes desafiadores (Ultralytics, 2020). A YOLOv8 mantém a abordagem de dividir a imagem em uma grade de células, mas melhora a forma como as previsões de localização e classe são feitas, resultando em uma detecção ainda mais rápida e precisa (TERVEN et al., 2023).

Embora esses avanços tenham contribuído significativamente para a detecção de objetos em cenários de segurança, ainda existem desafios a serem superados (Lampier, 2022). A adaptação a diferentes condições ambientais, como variações de iluminação, climas e ângulos de visão, bem como a capacidade de reconhecer uma ampla gama de tipos de armas em diferentes contextos, continua sendo uma preocupação (Gong, 2023). Portanto, a integração de sistemas inteligentes que utilizam a YOLOv8 em um Espaço Inteligente representa uma solução promissora para enfrentar os desafios da segurança pública moderna (Tang, 2024).

2.2 Optuna E Otimização

A otimização de modelos de detecção de objetos é crucial para melhorar a eficiência, a precisão e a aplicabilidade em cenários do mundo real. Uma das principais abordagens para otimização é o ajuste de hiperparâmetros, que envolve a modificação de variáveis como a taxa de aprendizado, o número de camadas e o tamanho do batch. Essa tarefa é fundamental para melhorar o desempenho do modelo e pode ser realizada através de técnicas como busca em grade, busca aleatória e algoritmos de otimização Bayesiana, que ajudam a encontrar as melhores configurações para um modelo específico.

Dentre essas ferramentas, o Optuna se destaca como uma plataforma eficiente para a otimização de hiperparâmetros. O Optuna é uma biblioteca de otimização que utiliza algoritmos de otimização adaptativa, permitindo buscas eficientes em espaços de hiperparâmetros complexos (Akiba, 2019). Em vez de explorar todas as combinações possíveis, o Optuna emprega um método conhecido como TPE (Tree-structured Parzen Estimator), que aborda o problema de otimização por meio de uma modelagem probabilística (LAI et al., 2023). O método TPE modela a função de perda e identifica as configurações que provavelmente resultarão em um melhor desempenho, dividindo as amostras em duas distribuições com base nos valores de desempenho dos modelos (função de perda), representadas por $l(x)$ e $g(x)$ (LAI et al., 2023).

O método constroi essas distribuições para minimizar a função de perda, definindo $l(x)$ como a distribuição dos hiperparâmetros que geraram perdas inferiores a um certo

limiar e $g(x)$ como a distribuição dos hiperparâmetros que excederam esse limiar. A distribuição posterior dos hiperparâmetros, $p(x|y)$, é então modelada como:

$$p(x|y) = \begin{cases} l(x) & \text{se } y < \gamma \\ g(x) & \text{se } y \geq \gamma \end{cases} \quad (1)$$

Onde γ representa um limiar adaptável que controla a exploração de novos valores versus a exploração de valores conhecidos que oferecem uma boa performance (LAI et al., 2023).

O objetivo do TPE é maximizar a razão de verossimilhança:

$$\frac{l(x)}{g(x)} \quad (2)$$

Essa razão tem a funcionalidade de buscar pelos hiperparâmetros mais promissores, onde valores altos indicam maior probabilidade de encontrar um conjunto de hiperparâmetros de alto desempenho.

Além disso, o Optuna adota técnicas de pruning para interromper experimentos com resultados abaixo de um certo nível de desempenho antes de serem completamente finalizados. Utilizando métricas intermediárias, como a precisão ou a perda parcial de cada iteração, o Optuna determina precocemente se um conjunto de hiperparâmetros alcançará um desempenho inferior, economizando tempo computacional (LAI et al., 2023).

Uma das principais vantagens do Optuna é sua capacidade de otimização dinâmica, que ajusta a busca em tempo real com base nas informações coletadas ao longo do processo de otimização. Isso não só acelera a busca por hiperparâmetros ideais, mas também reduz a quantidade de avaliações necessárias para encontrar configurações que maximizem a performance do modelo. A implementação do Optuna é particularmente

útil em cenários onde a eficiência e a rapidez são essenciais, como na detecção de objetos em tempo real em sistemas de segurança pública.

Essas estratégias de otimização, especialmente por meio do Optuna, não apenas melhoram o desempenho dos modelos, mas também possibilitam sua implementação em cenários práticos, onde a velocidade de processamento e a eficiência são críticas (AKIBA et al., 2019).

3. DESENVOLVIMENTO

3.1 Coleta e Preparação dos Dados

A coleta de dados foi realizada em colaboração entre os campi do IFES - Guarapari e Vitória, resultando em um conjunto robusto de imagens que reflete condições variadas de captura. Utilizando as câmeras dos espaços inteligentes nos Campi Guarapari e Vitória, foram capturadas imagens dentro de um laboratório de pesquisa em diferentes momentos do dia e sob diversas condições de iluminação, simulando com precisão ambientes reais de segurança pública. Essa diversidade de cenários foi fundamental para enriquecer o conjunto de dados e assegurar que o modelo treinado tivesse ampla adaptabilidade para uma aplicação em tempo real (COUTINHO et al., 2023).

No caso do dataset de Vitória, ele foi dividido em duas partes: uma contendo *data augmentation* e outra sem. O conjunto com *data augmentation* passou por transformações de imagem, como rotação, alteração de brilho, e outros efeitos, aumentando a variabilidade do dataset e, assim, permitindo que o modelo fosse mais robusto a diferentes condições de captura. Esse dataset com *data augmentation* foi utilizado para treinar um modelo sem otimização de hiperparâmetros. A outra parte, sem *data augmentation*, foi usada para treinar modelos que sofreram algum tipo de otimização.

Após a coleta, foi realizado um processo de rotulação manual para cada imagem, onde armas de fogo foram identificadas e delimitadas com caixas (bounding boxes). Esse processo de rotulação visou garantir a consistência e a qualidade dos dados rotulados,

preparando o conjunto de dados para as etapas de treinamento e validação do modelo de detecção. Para isso, 80% do dataset foi dedicado ao treinamento do modelo, enquanto os 20% restantes foram reservados para validação. Foram utilizadas ferramentas específicas para a rotulação, permitindo uma marcação precisa dos objetos de interesse e resultando em um dataset detalhado e adequado para avaliar a eficácia da solução desenvolvida.

3.2 Estrutura e Arquitetura da YOLO

A YOLO (You Only Look Once) é um modelo de deep learning amplamente utilizado para reconhecimento de objetos e padrões, e sua arquitetura é baseada em redes neurais convolucionais. Atualmente, é muito utilizada em trabalhos de visão computacional, pois apresenta desempenho superior em termos de velocidade em comparação a outros algoritmos, como R-CNN e Fast R-CNN (DIWAN et al., 2021). Essa eficiência se deve ao fato de que as imagens passam pelo detector apenas uma vez, enquanto nos métodos tradicionais esse processo ocorre várias vezes (Gi-Sang, 2022).

O funcionamento da YOLOv8 (You Only Look Once, versão 8) segue a abordagem tradicional de detecção de objetos, dividindo a imagem de entrada em uma grade de regiões. Cada região da grade gera múltiplas caixas delimitadoras (bounding boxes), associadas a valores de confiança e probabilidades de pertencimento a diferentes classes. Essa abordagem transforma a detecção de objetos em um único problema de regressão, permitindo que o modelo realize previsões rápidas e eficientes, sem a necessidade de etapas múltiplas como em outros modelos, como o Faster R-CNN (VIJAYAKUMAR & VAIRAVASUNDARAM, 2024).

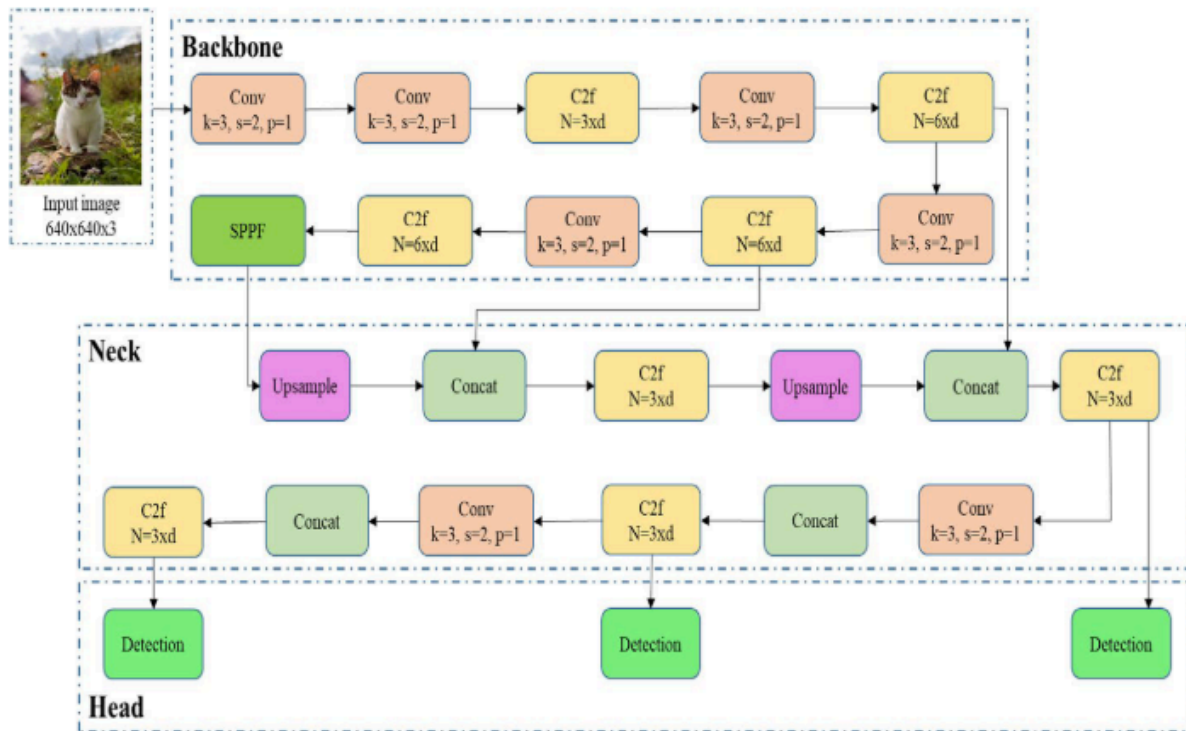


Figura 1. Arquitetura YOLOv8 (VIJAYAKUMAR & VAIRAVASUNDARAM, 2024).

A estrutura da YOLOv8 é dividida em duas partes principais: *backbone* e *head* (Hussain, 2023). A *backbone* é responsável pela extração das características da imagem de entrada. No caso da YOLOv8, uma rede convolucional profunda é utilizada para capturar informações em várias escalas. Essa capacidade de extrair características em diferentes níveis de detalhamento é fundamental para a detecção de objetos de tamanhos variados. A *backbone* é composta por camadas convolucionais seguidas de funções de ativação, como ReLU, e camadas de normalização, que estabilizam o processo de aprendizado e ajudam na ativação dos neurônios (Hussain, 2023). Essa arquitetura foi projetada para ser altamente eficiente, equilibrando precisão e velocidade, algo crucial em aplicações de detecção de objetos em tempo real (TERVEN et al., 2023).

A *head* da YOLOv8 é a parte responsável por gerar as previsões finais. Ela utiliza camadas totalmente conectadas para transformar as características extraídas pela *backbone* em saídas, que incluem as coordenadas das caixas delimitadoras e as classes associadas aos objetos detectados. A YOLOv8 introduz melhorias nessa parte, como o ajuste dinâmico das dimensões das caixas, utilizando uma abordagem baseada em ancoragem. Essa estratégia permite ajustar automaticamente as dimensões das caixas

delimitadoras de acordo com os objetos detectados, o que melhora a precisão das previsões, especialmente em cenários dinâmicos e variados (VIJAYAKUMAR & VAIRAVASUNDARAM, 2024).

Além disso, a YOLOv8 se diferencia pela flexibilidade oferecida em termos de configurações arquiteturais. Ela apresenta versões com diferentes tamanhos de rede, como Small (s), Medium (m) e Large (l). A principal diferença entre essas versões está na quantidade de parâmetros da rede: a arquitetura Small possui menos parâmetros, enquanto a Large contém o maior número, permitindo que o modelo Large tenha maior capacidade de aprender padrões mais complexos. Essa flexibilidade possibilita a escolha do modelo mais adequado para diferentes necessidades de aplicação, equilibrando desempenho e recursos computacionais. Nesse contexto, a arquitetura Medium oferece um bom compromisso entre os dois extremos (Ultralytics, 2020).

A implementação da YOLOv8 permite a execução de três scripts distintos: um para o treinamento do modelo, outro para a validação e o terceiro para o reconhecimento de objetos. Esses scripts, juntamente com a arquitetura escalável, tornam a YOLOv8 uma solução poderosa para detecção de objetos em tempo real, sendo amplamente utilizada em sistemas de monitoramento e segurança pública (TERVEN et al., 2023).

3.3 Arquitetura da YOLO v8 via Arquivo YAML

O arquivo de configuração YAML de um modelo YOLOv8 desempenha um papel crucial na definição da arquitetura e no controle dos parâmetros que orientam o treinamento e a performance do modelo. Ele oferece uma interface flexível para customizar a rede neural, ajustando a profundidade, largura e outras especificações da arquitetura, permitindo que o modelo seja adaptado a uma tarefa de detecção específica, como a detecção de armas de fogo. O arquivo é composto por parâmetros gerais que definem o tamanho da rede e pela configuração das duas partes principais da arquitetura: o *Backbone* e o *head* (Bo, 2024) (Ultralytics, 2020).

```

nc: 1

depth_multiple: 0.33
width_multiple: 0.50
max_channels: 512

backbone:
  - [-1, 1, Conv, [64, 3, 2], ]
  - [-1, 1, Conv, [128, 3, 2], ]
  - [-1, 3, C2f, [128, True], ]
  - [-1, 1, Conv, [256, 3, 2], ]
  - [-1, 6, C2f, [256, True], ]
  - [-1, 1, Conv, [512, 3, 2], ]
  - [-1, 6, C2f, [512, True], ]
  - [-1, 1, Conv, [1024, 3, 2], ]
  - [-1, 3, C2f, [1024, True], ]
  - [-1, 1, SPPF, [1024, 5], ]

head:
  - [-1, 1, nn.Upsample, ['None', 2, 'nearest'], ]
  - [[-1, 6], 1, Concat, [1], ]
  - [-1, 3, C2f, [512], ]
  - [-1, 1, nn.Upsample, ['None', 2, 'nearest'], ]
  - [[-1, 4], 1, Concat, [1], ]
  - [-1, 3, C2f, [256], ]
  - [-1, 1, Conv, [256, 3, 2], ]
  - [[-1, 12], 1, Concat, [1], ]
  - [-1, 3, C2f, [512], ]
  - [-1, 1, Conv, [512, 3, 2], ]
  - [[-1, 9], 1, Concat, [1], ]
  - [-1, 3, C2f, [1024], ]
  - [[15, 18, 21], 1, Detect, ['nc'], ]

```

Figura 2. Arquitetura padrão de um modelo YOLOv8.

A figura 2 apresenta o arquivo de configuração padrão da arquitetura YOLOv8, onde são definidos os parâmetros de construção da rede neural, como o número de repetições das camadas, os filtros aplicados em cada estágio e a estrutura geral da rede.

O tamanho da rede é definido pelos parâmetros *depth_multiple* e *width_multiple*, que controlam, respectivamente, a profundidade e a largura da arquitetura. O *depth_multiple* ajusta o número de camadas da rede, enquanto o *width_multiple* determina o número de filtros em cada camada convolucional, influenciando diretamente a capacidade de aprendizado e a eficiência do modelo. Além disso, o parâmetro *max_channels* estabelece um limite para o número máximo de canais nas camadas convolucionais, evitando que a rede fique excessivamente grande e garantindo um equilíbrio entre desempenho e eficiência computacional. A Tabela 1 descreve cada parâmetro de forma detalhada.

Parâmetro	Descrição
nc	Número de classes que o modelo deve detectar. Nesse caso, é 1, o que significa que o modelo é para detectar uma única classe.
depth_multiple	Fator de multiplicação para a profundidade da rede (controla o número de camadas).
width_multiple	Fator de multiplicação para a largura da rede (controla o número de filtros nas camadas convolucionais).
max_channels	Número máximo de canais nas camadas convolucionais

Tabela 1: Descrição dos parâmetros de tamanho da rede

O *Backbone* é composto por uma sequência de camadas convolucionais e blocos especializados, como *C2f* e *SPPF*, que têm como objetivo extrair as características mais relevantes das imagens. A cada camada, o modelo realiza operações como convoluções e pooling, seguidas de transformações nas características extraídas. Cada operação é especificada no arquivo YAML, o que possibilita o controle preciso sobre como as informações visuais são processadas pela rede. A Tabela 2 mostra uma descrição de cada bloco especializado e sua devida função.

Componente	Descrição
Conv	Camadas responsáveis por extrair características simples, como bordas e texturas, aplicando filtros em diferentes resoluções. Utiliza parâmetros como número de filtros, tamanho do kernel e stride.
C2f	Bloco modular que utiliza múltiplas convoluções e conexões residuais para melhorar a captura de características e reduzir o risco de overfitting.
SPPF	Camada que aplica pooling espacial para capturar informações de diferentes escalas.

Tabela 2: Descrição dos parâmetros de tamanho da rede

O *head*, por sua vez, realiza a parte final da detecção, onde as características extraídas pelo *Backbone* são refinadas para prever a localização e as classes dos objetos. Este componente envolve operações como upsampling, que aumenta a resolução das características, e concatenação, que combina as saídas de diferentes camadas para permitir ao modelo utilizar informações de múltiplas escalas. A camada final de detecção é definida com a operação *Detect*, que gera as previsões de classe e localização dos objetos. A Tabela 3 traz uma descrição de componente do *head*.

Componente	Descrição
Upsample (nn.Upsample)	Aumenta a resolução das características extraídas pelo <i>Backbone</i> . Utiliza a operação de upsampling para refinar a localização dos objetos
Concatenação (Concat)	Combina as saídas de diferentes camadas de resoluções variadas para capturar características em várias escalas.
Detect	Realiza a detecção final, combinando as saídas das diferentes resoluções (P3, P4, P5) e gerando as caixas delimitadoras, classes dos objetos e a confiança nas previsões.

Tabela 3: Descrição dos componentes do *head*

Em resumo, o arquivo de configuração YAML da YOLOv8 permite um controle detalhado sobre a arquitetura do modelo, oferecendo a flexibilidade necessária para adaptá-lo a tarefas de detecção específicas. Isso é especialmente útil quando se busca equilibrar a complexidade do modelo com a necessidade de obter previsões rápidas e precisas, adequando o modelo para aplicações em tempo real, como sistemas de videomonitoramento ou segurança.

3.4 Otimização de Hiperparâmetros e maximização de métricas

A otimização de hiperparâmetros desempenha um papel crucial na melhoria do desempenho de modelos de aprendizado de máquina, especialmente na detecção de objetos. Para o presente trabalho, foram ajustados parâmetros específicos, como a taxa

de aprendizado, momentum, weight decay, número de camadas de convolução e o tipo de rede convolucional. Essas modificações foram realizadas exclusivamente na *backbone*, uma vez que a YOLO é projetada para otimizar a detecção e classificação na *head*. O foco foi aprimorar a extração de características a partir das redes convolucionais, visando aumentar a eficácia na detecção de armas de fogo em ambientes de segurança pública.

A biblioteca Optuna foi utilizada para facilitar esse processo de otimização. Optuna permite a busca eficiente no espaço dos hiperparâmetros, aplicando técnicas de otimização bayesiana. O processo envolve a definição de um espaço de busca onde os hiperparâmetros são configurados, e cada configuração é avaliada em iterações sucessivas. Essa abordagem possibilita a identificação de combinações de parâmetros que melhoram o desempenho do modelo.

Após as modificações, o modelo foi submetido a um treinamento, seguido de uma fase de validação. A avaliação do desempenho foi realizada com base em métricas relevantes, como *precision*, *recall* e *mAP@50*. Essas métricas proporcionam uma visão abrangente da eficácia do modelo, permitindo uma análise detalhada de sua capacidade de detecção de armas de fogo.

As métricas de avaliação são fundamentais para medir o desempenho de modelos de detecção de objetos, como a YOLO (You Only Look Once). Entre as principais métricas estão a *Precision*, o *Recall* e o *mAP@50*, que fornecem uma visão detalhada sobre a capacidade do modelo de identificar e localizar corretamente os objetos em uma imagem.

A *Precision* avalia a proporção de predições corretas, chamadas de verdadeiros positivos (TP), em relação ao total de predições realizadas pelo modelo, incluindo os falsos positivos (FP). Matematicamente, é definida como:

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

Um valor alto de precisão indica que o modelo realiza predições confiáveis, com poucas detecções incorretas (VIJAYAKUMAR & VAIRAVASUNDARAM, 2024).

Por outro lado, o *Recall* mede a capacidade do modelo de identificar todos os objetos presentes na imagem, sendo calculado como:

$$Recall = \frac{TP}{TP + FN} \quad (4)$$

Na equação 4, FN são os falsos negativos, ou seja, objetos que estão presentes na imagem, mas que o modelo não conseguiu detectar. Um alto *recall* indica que o modelo identifica a maioria dos objetos reais, minimizando as omissões.

Essas métricas dependem de um critério para determinar se uma detecção é correta. Para isso, utiliza-se a métrica IoU (Intersection over Union), que mede a sobreposição entre a caixa delimitadora predita pelo modelo e a caixa real (ground truth). O IoU é calculado como:

$$IoU = \frac{\text{Intersection Area}}{\text{Union Area}} \quad (5)$$

Se o IoU entre uma detecção e o ground truth for maior ou igual a um limite predefinido, como 0.5, a detecção é classificada como um verdadeiro positivo; caso contrário, é um falso positivo (Terven et al., 2023).

O *mAP@50* (Mean Average Precision at IoU = 0.5) combina os conceitos de precisão e *recall*, fornecendo uma única métrica que avalia o desempenho geral do modelo em todas as classes e em diferentes limiares de confiança. Para cada classe, calcula-se a Average Precision (AP), que é a média da precisão em diferentes valores de *recall*. O “mAP” é então obtido como a média das APs de todas as classes. No caso específico do *mAP@50*, considera-se um limiar de IoU fixo de 0.5 para determinar a correspondência entre predições e ground truths. Essa métrica é amplamente usada para avaliar modelos de detecção de objetos, pois oferece uma visão equilibrada do desempenho do modelo, considerando tanto a qualidade das detecções quanto a capacidade de localizar todos os objetos relevantes (VIJAYAKUMAR & VAIRAVASUNDARAM, 2024; TERVEN et al., 2023).

4. RESULTADOS E DISCUSSÕES

Nesta seção será apresentada uma análise abrangente do desempenho de três modelos da YOLOv8 em relação à detecção de armas de fogo, permitindo uma avaliação detalhada das melhorias introduzidas por meio da otimização de hiperparâmetros e modificações na arquitetura. Os três modelos da YOLOv8 utilizados neste estudo diferem em suas abordagens de treinamento e na complexidade de suas arquiteturas. O Modelo 1 emprega a YOLO de forma tradicional, utilizando uma arquitetura Large, que possui um número maior de parâmetros e, portanto, uma capacidade robusta de detecção. No entanto, essa complexidade pode levar a uma maior exigência computacional, o que pode não ser ideal para aplicações em tempo real.

Em contraste, o Modelo 2 foi otimizado com hiperparâmetros ajustados por meio do Optuna, e utiliza uma arquitetura Small. Essa configuração menor reduz o tempo de processamento e melhora a eficiência, sem sacrificar significativamente a precisão na detecção. O ajuste cuidadoso dos hiperparâmetros permite que o Modelo 2 mantenha um desempenho competitivo, mesmo com um número reduzido de parâmetros.

O Modelo 3, por sua vez, também é baseado na arquitetura Small, mas vai além ao incluir modificações na *backbone*, como mudanças nos parâmetros das camadas de convolução e no tipo de rede convolucional. Essas adaptações foram otimizadas e personalizadas utilizando o Optuna, uma ferramenta de otimização baseada em busca bayesiana que ajusta os hiperparâmetros da arquitetura para alcançar o melhor desempenho possível. O objetivo dessas modificações foi melhorar a extração de características e adaptar o modelo para os cenários específicos de detecção de armas de fogo. Assim, embora todos os três modelos tenham como objetivo a detecção, suas diferenças em termos de complexidade e ajustes tornam cada um deles adequado para contextos específicos, especialmente em termos de eficiência e velocidade em aplicações práticas.

A figura 3 mostra a nova configuração da arquitetura do Modelo 3 com hiperparâmetros e componentes obtidos através do Optuna.

```
nc: 1

depth_multiple: 0.33
width_multiple: 0.50
max_channels: 512

backbone:
- [-1, 1, Conv, [64, 3, 2], ]
- [-1, 1, Conv, [128, 3, 2], ]
- [-1, 3, C2f, [128, True], ]
- [-1, 3, Conv, [256, 3, 2], ]
- [-1, 6, C2f, [256, True], ]
- [-1, 1, Conv, [512, 3, 2], ]
- [-1, 6, C2f, [512, True], ]
- [-1, 2, Conv, [1024, 3, 2], ]
- [-1, 3, SPPF, [2048, 5], ]
- [-1, 6, C2f, [2048, True], ]
- [-1, 3, SPPF, [2048, 5], ]

head:
- [-1, 1, nn.Upsample, ['None', 2, 'nearest'], ]
- [[-1, 6], 1, Concat, [1], ]
- [-1, 3, C2f, [512], ]
- [-1, 1, nn.Upsample, ['None', 2, 'nearest'], ]
- [[-1, 4], 1, Concat, [1], ]
- [-1, 3, C2f, [256], ]
- [-1, 1, Conv, [256, 3, 2], ]
- [[-1, 12], 1, Concat, [1], ]
- [-1, 3, C2f, [512], ]
- [-1, 1, Conv, [512, 3, 2], ]
- [[-1, 9], 1, Concat, [1], ]
- [-1, 3, C2f, [1024], ]
- [[15, 18, 21], 1, Detect, ['nc'], ]
```

Figura 3: Arquitetura do Modelo 3 com parâmetros obtidos pelo Optuna.

As Figuras 4, 5 e 6 ilustram o desempenho de cada modelo ao longo de 100 épocas de treinamento em diferentes métricas.

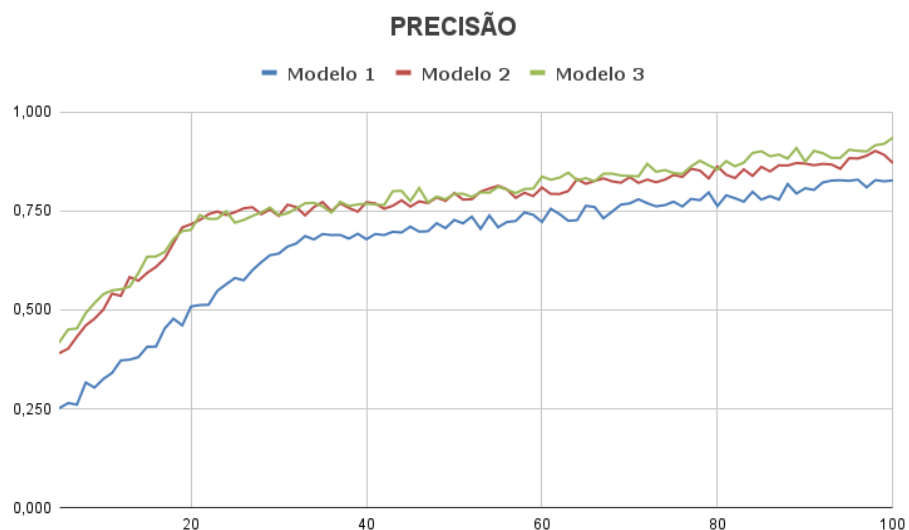


Figura 4. Precisão ao longo das épocas de treinamento para cada modelo

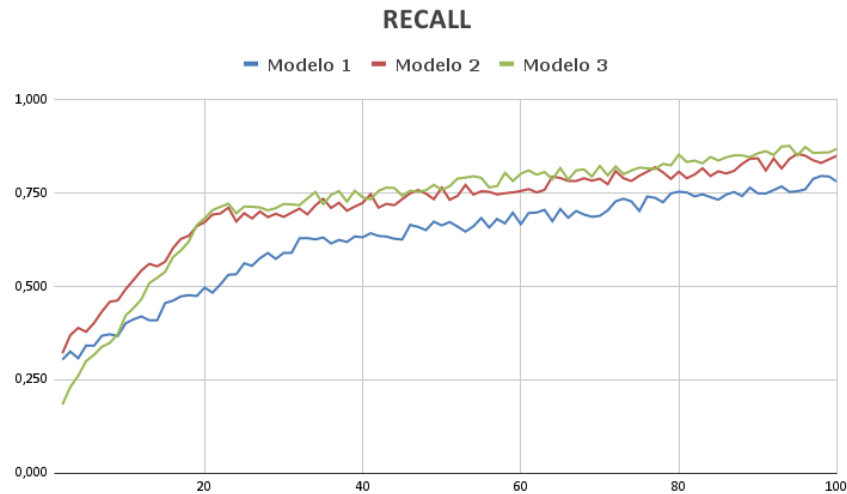


Figura 5. *Recall* ao longo das épocas de treinamento para cada modelo

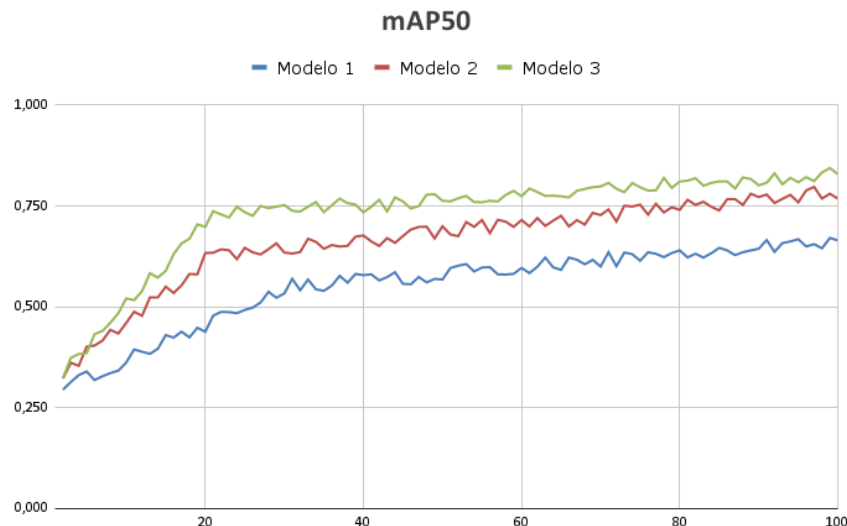


Figura 6. *mAP@50* ao longo das épocas de treinamento para cada modelo

A Figura 4 apresenta a precisão ao longo das épocas, demonstrando que tanto os Modelos 2 quanto 3 partem com uma precisão maior em comparação ao Modelo 1. Após aproximadamente 30 épocas, a diferença de precisão entre o Modelo 1 e os demais diminui, embora o Modelo 3 mantenha uma leve vantagem sobre o Modelo 2.

A Figura 5 explora o *recall* ao longo das mesmas épocas. Observa-se que, embora o Modelo 3 inicie com um *recall* inferior aos Modelos 1 e 2, ele rapidamente ultrapassa os outros modelos após um número reduzido de épocas. Essa capacidade de adaptação do

Modelo 3 pode ser um indicativo de sua robustez em ambientes de detecção, onde a velocidade e a precisão são cruciais. O fato de o Modelo 3 conseguir melhorar seu desempenho tão rapidamente destaca a eficácia das otimizações realizadas na arquitetura, refletindo um potencial significativo para aplicações práticas em segurança pública.

Por fim, a Figura 6 apresenta o $mAP@50$, onde o Modelo 3 demonstra um desempenho superior em relação ao Modelo 2, ambos superando o Modelo 1. Essa métrica, que avalia a precisão média em diferentes limiares, é fundamental para entender a eficácia geral do modelo na detecção de armas de fogo, pois fornece uma visão clara de como cada modelo responde à tarefa proposta.

As Tabelas 4 e 5 comparam os resultados obtidos na pesquisa e os resultados de um estudo de referência. A Tabela 4 apresenta a avaliação dos três modelos propostos nesta pesquisa, enquanto a Tabela 5 mostra os resultados de um autor de referência, que avaliou redes com três diferentes tamanhos (small, medium e large), permitindo uma comparação direta das performances entre as diferentes abordagens e escalas de modelos.

Modelo 1	<i>Precision</i>	0,829
	<i>Recall</i>	0,796
	<i>mAP@50</i>	0,671
Modelo 2	<i>Precision</i>	0,902
	<i>Recall</i>	0,856
	<i>mAP@50</i>	0,798
Modelo 3	<i>Precision</i>	0,935
	<i>Recall</i>	0,881
	<i>mAP@50</i>	0,844

Tabela 4. Métricas obtidas da validação de cada modelo

Small	<i>Precision</i>	0,903
	<i>Recall</i>	0,888
	<i>mAP@50</i>	0,875
Medium	<i>Precision</i>	0,909
	<i>Recall</i>	0,889
	<i>mAP@50</i>	0,881
Large	<i>Precision</i>	0,913
	<i>Recall</i>	0,899
	<i>mAP@50</i>	0,888

Tabela 5. Métricas obtidas da validação a partir de técnicas tradicionais (COUTINHO et al., 2023).

A comparação entre os resultados da presente pesquisa e os obtidos por *Coutinho et al.* (2023) evidencia diferenças importantes nas abordagens utilizadas e nos desempenhos alcançados. Nesta pesquisa, foram otimizados hiperparâmetros e a arquitetura da YOLOv8 *small*, utilizando apenas 100 épocas de treinamento, enquanto *Coutinho et al.* empregou métodos tradicionais, avaliando as versões *small*, *medium* e *large* do modelo, com um tempo de treinamento de 300 épocas.

Na versão *small*, o modelo 3 desta pesquisa demonstrou um desempenho competitivo. Com uma *precision* de 0,935, superou a versão *small* de *Coutinho et al.* (0,903), destacando uma maior capacidade de prever corretamente as classes identificadas. No *recall*, o modelo 3 alcançou 0,881, valor ligeiramente inferior ao *recall* de 0,888 do modelo *small* de *Coutinho et al.*, mas ainda assim muito próximo, indicando uma taxa de detecção de objetos consistente. No *mAP@50*, o modelo 3 obteve 0,844, enquanto *Coutinho et al.* alcançou 0,875, uma diferença que reflete o menor número de épocas e o foco em arquiteturas mais compactas nesta pesquisa.

Já os modelos *medium* e *large* de *Coutinho et al.* apresentaram aumentos esperados nos resultados, com o *medium* atingindo um $mAP@50$ de 0,881 e o *large* de 0,888. No entanto, mesmo comparando com essas versões, a *precision* do modelo 3 (0,935) foi superior, superando inclusive o modelo *large* (0,913). Isso reforça a eficácia das otimizações realizadas, que permitiram alcançar alta *precision* com uma arquitetura mais simples e eficiente.

Apesar dos valores de *recall* do modelo 3 serem ligeiramente menores em comparação aos de *Coutinho et al.*, a proximidade entre eles demonstra que as otimizações realizadas não comprometeram a taxa de detecção de objetos, mantendo o modelo altamente competitivo. Essa análise destaca que, com menos épocas de treinamento e um modelo mais enxuto, foi possível obter resultados comparáveis e, em alguns aspectos, superiores aos alcançados por arquiteturas maiores e mais complexas, tornando o modelo 3 particularmente atrativo para aplicações em tempo real, onde o custo computacional é crítico.

5. CONCLUSÃO

A partir dos resultados obtidos, conclui-se que foi possível desenvolver um modelo de detecção de armas de fogo que, além de apresentar um tamanho reduzido, foi otimizado em termos de hiperparâmetros e arquitetura, resultando em um desempenho competitivo em comparação com abordagens tradicionais. A utilização do Optuna para a otimização foi fundamental, permitindo que as versões menores dos modelos, particularmente os Modelos 2 e 3, alcançassem elevada precisão e valores de *recall* próximos aos modelos tradicionais, mesmo com um número reduzido de épocas de treinamento.

A relevância de um modelo compacto é especialmente evidente em aplicações práticas, como segurança pública. Modelos menores demandam menos recursos computacionais, viabilizando sua implementação em dispositivos com capacidade limitada, como câmeras de segurança e sistemas de monitoramento em tempo real. Além

disso, a leveza do modelo contribui para tempos de resposta mais rápidos, um aspecto essencial em situações onde a detecção imediata é indispensável. Dessa forma, esta pesquisa não apenas avança o estado da arte em detecção de objetos, mas também oferece uma base robusta para o desenvolvimento de soluções mais eficientes e eficazes voltadas para a segurança pública, conciliando alto desempenho com viabilidade operacional.

6. REFERÊNCIAS

Miranda, Zil. Política de Ciência, Tecnologia e Inovação para a Segurança Pública. São Paulo. Revista Brasileira de Segurança Pública, 2012.

Coelho, Danilo Santa Cruz. Inteligência Artificial em Justiça e Segurança Pública: Exemplos e Recomendações para Políticas Públicas. Rio de Janeiro, Brasil. Instituto de Pesquisa Econômica Aplicada, 2024.

Loureiro, Bruno Medeiros. Cidades Inteligentes e Segurança Cidadã: Uma Análise do Funcionamento do Cerco Inteligente de Vitória. Vila Velha, Brasil: Universidade de Vila Velha, 2021.

S. Al-E'mari, Y. Sanjalawe and H. Alqudah, "Integrating Enhanced Security Protocols with Moving Object Detection: A Yolo-Based Approach for Real-Time Surveillance," 2024 2nd International Conference on Cyber Resilience (ICCR), Dubai, United Arab Emirates, 2024.

Lampier, L. C., Coelho, Y. L., Caldeira, E. M. O., & Bastos-Filho, T. F. (2022). A Deep Learning Approach to Estimate the Respiratory Rate from Photoplethysmogram. Ingenius, 2022.

Yan Gong, Suzhou Institute of Biomedical Engineering and Technology, Chinese Academy of Sciences, et al. "Swin-Transformer-Based YOLOv5 for Small-Object Detection in Remote Sensing Images", Sensors, Vol. 23, No. 7, 2023

Tang, Qiang; Li, Xiangwei; Xie, Meilin & Zhen, Jialiang. Intelligent Space Object Detection Driven by Data from Space Objects. University of Chinese Academy of Sciences, China, 2024.

Gi-Sang Choi, Department of Electrical and Computer Engineering, Graduate School, University of Seoul, et al. "Improved YOLOv5: Efficient Object Detection Using Drone Images under Various Conditions", Applied Sciences, Vol. 12, 2022.

Fan, Lujie. High-Precision Tomato Maturity Detection Using a Genetic Algorithm Optimized Swin-YOLO Network. Shouguang, China: College of Computer, Weifang University of Science and Technology, 2024.

Vijayakumar, Ajantha; Vairavasundaram, Subramaniaswamy. YOLO-based Object Detection Models: A Review and its Applications. Thanjavur, India: School of Computing, SASTRA Deemed University, 2024.

Diwan, Tausif; Anirudh, G.; Tembhurne, Jitendra V. Object detection using YOLO: challenges, architectural successors, datasets and applications. Jaipur, Rajasthan, India: Central University of Rajasthan, 2023.

Lai, Jung-Pin; Lin, Ying-Lei; Lin, Ho-Chuan; Shih, Chih-Yuan; Wang, Yu-Po; Pai, Ping-Feng. Tree-Based Machine Learning Models with Optuna in Predicting Impedance Values for Circuit Analysis. Puli Nantou, Taiwan: National Chi Nan University, 2023.

Hussain, Muhammad. YOLO-v1 to YOLO-v8, the Rise of YOLO and Its Complementary Nature toward Digital Manufacturing and Industrial Defect Detection. Huddersfield, Reino Unido: University of Huddersfield, 2023.

Terven, Juan; Córdova-Esparza, Diana-Margarita; Romero-González, Julio-Alejandro. A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. Queretaro, México: Instituto Politécnico Nacional, CICATA-Qro, 2023.

Zhao, Kai; Lu, Ruitao; Wang, Siyu; Yang, Xiaogang; Li, Qingge; Fan, Jiwei. ST-YOLOA: a Swin-transformer-based YOLO model with an attention mechanism for SAR ship detection under complex background. Xi'an, China: Department of Automation, Rocket Force University of Engineering, 2023.

Coutinho, Loureni; Soprani, Douglas; Canuto, Clebeson; do Carmo, Alexandre; Amigo, Bruno; Rampinelli, Mariana; Ribeiro, Rodolfo; Pereira, Rafael. Videomonitoramento em Espaços Inteligentes: Uma Aplicação de Segurança Pública em Identificação de Armas de Fogo. Instituto Federal do Espírito Santo, Campus Guarapari: Guarapari, Brasil, 2022.

Ozaki, Yoshihiko; Tanigaki, Yuki; Watanabe, Shuhei; Nomura, Masahiro; Onishi, Masaki. Multiobjective Tree-Structured Parzen Estimator Yoshihiko. Journal of Artificial Intelligence Research, v. 73, 2022.

Junos, Mohamad Haniff; Dahari, Mahidzal; Khairuddin, Anis Salwa Mohd; Thannirmalai, Subbiah. An optimized YOLO-based object detection model for crop harvesting system. Kuala Lumpur, Malaysia: University of Malaya, 2021.

Ultralytics. YOLO by Ultralytics. [S. l.]: Ultralytics, 2023. Disponível em: <https://ultralytics.com>. Acesso em: 10 out. 2024.

Akiba, Takuya; Sano, Shotaro; Yanase, Toshihiko; Ohta, Takeru; Koyama, Masanori. Optuna: A Next-generation Hyperparameter Optimization Framework. In: ACM SIGKDD INTERNATIONAL CONFERENCE ON KNOWLEDGE DISCOVERY AND DATA MINING, 25., 2019. Proceedings [...]. [S. l.]: ACM, 2019.

Gomes, M. A. Segurança pública brasileira: desafios e propostas de melhorias. Relatório técnico, 2019. Disponível em: <https://blog.ipog.edu.br/educacao/seguranca-publica/>. Acesso em: 7 out. 2024.

IPEA. Óbitos por armas de fogo. Relatório técnico, 2024. Disponível em: <https://www.ipea.gov.br/atlasviolencia/dados-series/180>. Acesso em: 7 nov. 2024.

Bo. Liu, "YOLO-M: An Efficient YOLO Variant with MobileOne Backbone for Real-Time License Plate Detection," 2024 5th International Seminar on Artificial Intelligence, Networking and Information Technology (AINIT), Nanjing, China, 2024.